MYSchool ERP Master Blueprint Canonical Technical Handover Dossier

Print / Export as PDF

INSTITUTIONAL TECHNICAL BLUEPRINT

MYSchool ERP Specification

Multi-Branch Architecture, High-Concurrency Workflows & Antigravity Operational Directives

🏛️ **Institution:** JSS School

⚡ **Engine:** Next.js 15+ & Supabase (Supavisor)

🚀 **Cache:** Upstash Redis (>90% Hit Ratio)

🔐 **Auth:** NextAuth.js (Auth.js v5)

🎯 **Concurrency:** 5,000+ Concurrent Requests

📌 **Status:** Client-Ready & Vibe-Coding Locked

01 Product Overview & Strategic Problem

MYSchool PRD — Core Vision & Guardrails

🎯 The Downside of Not Having This System

- **Delayed Duty-of-Care Alerts:** Manual 8:30 AM paper registers delay time-sensitive absence alerts to parents.
- **Accounting Ledger Drift:** Unverified digital fee receipts and bounced transfers cause ledger discrepancies.
- **Lost Statutory Records:** Uncoordinated departures risk the permanent loss of regulatory exit certificates.
- **Morning Roll-Call Crashes:** Unpooled serverless queries crash under 100+ concurrent classroom spikes.

🏁 What Success Looks Like

- **Digitizes, automates, and connects** all daily academic and administrative operations across JSS School.
- **100+ classroom teachers log 40 students** simultaneously in under 60 seconds with single-payload writes (~10ms).
- **Fee dues deduct only** after campus bursar verifies physical bank statement reconciliation.
- **Mandatory verified exit certificates** (TC, Character, No-Dues) strictly guard permanent student deletions.
- **Students retain lifetime access** to download official historical transcripts and report cards.

ATTRIBUTE	SPECIFICATION FOR JSS SCHOOL
Institutional Hierarchy	Central Trust (JSS) → Physical Campuses (Main Campus, North Campus, East Campus) → Academic Wings (Primary, Middle, Secondary, Senior Secondary).

ATTRIBUTE	SPECIFICATION FOR JSS SCHOOL
Primary Platform	Responsive Full-Stack Web Application (Optimized for Desktop Office Terminals & Mobile Teacher Browsers).
Core Tenancy Model	Shared Multi-Branch PostgreSQL schema with dynamic URL scoping (`/branches/[branchSlug]/[departmentCode]/...`).

02 Core Functional Modules & Workflows

Seven Non-Negotiable Operational Pillars

⚡ High-Speed Attendance Roll-Call

Teachers load class rosters in one click. Single-action submit dispatches one array payload (`createMany`) committing 40 records in ~10ms. Real-time webhook fires SMS/push alerts for unexcused absences.

💳 Manual Fee UTR Verification

Shows fee dues and UPI intents. Validates UTR uniqueness against duplicates and supports partial payments. **Balance is strictly untouched** until campus bursars audit physical bank statements.

📝 Student Leave Applications

Parents submit leave requests with date spans and doctor attachments, routed to Class Teachers. Approved leaves automatically flag calendar roll-calls as locked `EXCUSED_LEAVE`.

👨‍👩‍👦 Multi-Ward Single Parent Hub

Dedicated single-parent family model (no dual-parenting complexity). One login manages multiple wards across campuses with persistent top navbar switching.

🤖 Smart Substitution Assistant (Timetables)

Real-time collision engine detects teacher absences, queries free period slots in the same department wing, and generates 1-click ranked faculty matches without mutating the master timetable.

⚖️ Dual-Signoff Exam Grade Moderation

Subject teachers submit student marks as Draft; Wing Coordinators / HODs inspect class curves and averages before signoff. Grades publish to parents only upon final approval.

📊 Role-Adaptive Clean Dashboards

Zero-clutter interface tailoring: Teachers see attendance matrices and period schedules; Bursars see split-screen bank audits; Parents see 1-tap sibling switchers and fee cards.

⚡ Read-Heavy Caching via Redis (Upstash)

Master calendars, department topology, and subject lists cached at the edge (1h–24h TTL) with >90% hit ratio, offloading 80% of database load and protecting connection pool slots.

🔒 Exit-Certificate Deletion Barrier

Permanent student deletions require typed `admission_no` confirmation and are hard-blocked until 4 statutory certificates (TC, Character, Migration, No-Dues) are verified.

📄 Background Batch PDF Engine

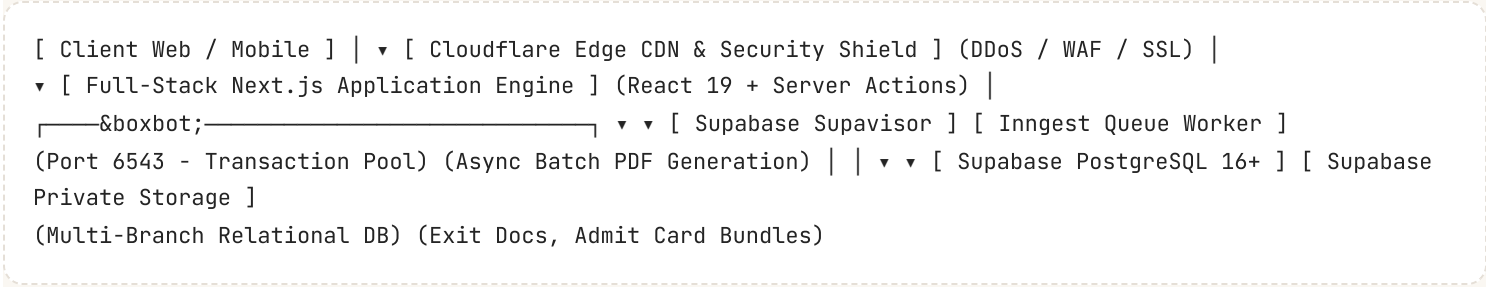
Buzzer Token pattern hands back a `jobId` in ~30ms. Inngest renders 2,000 admit cards and term receipts directly to Supabase Storage with 24-hour pre-signed download links.

📅 Academic Cycles & Lifetime Historical Transcripts

JSS School defines annual academic cycles. Past attendance archives to cold storage while **students retain permanent lifetime access to download official transcripts**. Staff attendance is tracked with daily timestamps.

03 System Architecture & Concurrency Strategy

Engineering Stability for 5,000+ Users



1. Supavisor Pooling

Solves serverless socket exhaustion. 500 concurrent teacher requests multiplex across **15–25 physical database connections** via transaction pooling on port 6543.

2. Direct Pre-Signed Uploads

Large student admission PDFs and exit certificates upload directly from client browsers to Supabase Storage, **bypassing Next.js server memory and bandwidth**.

3. Asynchronous PDF Jobs

Inngest handles mass admit cards and fee receipts off-thread. Administrators receive an active progress bar and a **24-hour signed download URL** upon job completion.

FAILURE SCENARIO	ROOT CAUSE & REAL-WORLD RISK	TECHNICAL HANDLING STRATEGY & SAFEGUARD
8:30 AM Attendance Surge	**Serverless Socket Exhaustion:** 100+ teachers submitting simultaneously spins up hundreds of serverless threads, depleting PostgreSQL's 100-connection limit and crashing with `500 Internal Error`.	**Bulk Payload + Supavisor Pooling:** Single array payload of 40 students committed in ~10ms via Prisma `createMany`. Supavisor (Port 6543) transaction mode holds sockets only during active write, multiplexing 500+ callers into **15–25 physical connections**.
Batch PDF Generation	**HTTP 504 Gateway Timeout:** Compiling 500+ admit cards with student photos and barcodes takes 1–3 minutes, exceeding the 60s reverse proxy / browser timeout limit.	**Inngest Async Token Pattern:** Offloaded to Inngest with instant 200 response and `jobId` in ~30ms. Headless worker renders in chunks, streams PDFs to Supabase Storage, and issues a 24h pre-signed download link.

FAILURE SCENARIO	ROOT CAUSE & REAL-WORLD RISK	TECHNICAL HANDLING STRATEGY & SAFEGUARD
Invalid Fee UTR Number	**Ledger Drift & Fraud:** Naive systems deduct student fee dues immediately on submission. If bank transfer fails or UTR is fake, accounting ledgers desynchronize.	**Two-Step Manual Audit Queue:** Submissions are tagged `PENDING_VERIFICATION`; **due balance remains untouched** until campus bursar audits bank records and clicks `[VERIFY & APPROVE]`.
Unverified Student Deletion	**Regulatory Liability & Lost Transcripts:** Deleting a student before statutory clearance violates education board mandates and destroys academic history.	**Statutory Exit Barrier Guard:** Hard programmatic check in deletion API. Verifies that Transfer Certificate (TC), Character Certificate, and No-Dues Slip exist with status `VERIFIED` before deletion.
Heavy Document Uploads	**Server Memory Exhaustion & Timeouts:** Uploading 15MB certificates through server threads exhausts Node.js RAM and triggers HTTP 504 timeouts on mobile.	**Presigned S3 Direct Uploads (`src/lib/storage.ts`):** Next.js signs a 60s HMAC token. Client browser streams file directly to S3 / Cloudflare R2, keeping server RAM at **0MB** and CPU at **0%**.
Mobile Payment Double-Clicks	**Duplicate Ledger Entries:** Impatient parents on flaky mobile connections double-tap "Submit", generating race conditions and duplicate UTR records.	**Strict Idempotency Engine (`src/lib/idempotency.ts`):** Client modal attaches a UUID. Server executes an atomic Redis lock (`SET key NX EX 60`), returning the cached pending response for subsequent taps.

04 Enterprise Data Model & Entity Relations

JSS School Relational Hierarchy Overview

ENTITY TABLE	KEY FIELDS & CONSTRAINTS	RELATIONAL PURPOSE
`organizations`	`id, name ("JSS School")`	Root institutional tier for group-wide executive leadership.
`branches`	`id, org_id, name, slug ("main", "north")`	Physical campuses with localized principals and staff.
`departments`	`id, branch_id, name, code ("PRIM", "PUC")`	Educational divisions mapped to distinct classes and HODs.
`users`	`id, branch_id, email, role, password_hash`	Role-based authentication across central and campus staff.
`students`	`id, user_id, class_id, admission_no, guardian_id`	Enrollee profile linked to guardian and academic class.
`fee_invoices`	`id, student_id, total_due, paid_amount, status`	Ledger tracking student fee schedules and verified balances.
`payment_submissions`	`id, invoice_id, utr_number, status, verified_by_id`	Manual transaction ID store audited by campus bursars.
`leave_applications`	`id, student_id, start_date, end_date, status`	Absence request store linked to class teacher approval loops.
`exit_certificates`	`id, student_id, certificate_type, file_url`	Mandatory barrier documents required before student purge.

05 Antigravity AI Agent Instructions (AGENTS.md)

Operational Laws for Autonomous Vibe Coding

1. Inspect →

Examine existing schema & folder boundaries

2. Plan →

Identify affected types & verify non-destructive rules

3. Implement →

Write minimal, isolated code in feature directory

4. Verify ✓

Run compiler check (`tsc --noEmit`)

✓ DO WORKING RULES

- ✓ Scope all queries by `branchId` AND `departmentId` .
- ✓ Use single-array `createMany` inserts for morning roll-calls.
- ✓ Import database singleton from `@/lib/db` (Supavisor pool).
- ✓ Require verified exit certificates before student deletion actions.
- ✓ Separate domain logic into `features/*/server/*.service.ts` .

✖ AVOID STRICT PROHIBITIONS

- ✖ Never execute `prisma db push --force-reset` or drop columns.
- ✖ Never execute unconstrained school-wide database lookups.
- ✖ Never instantiate `new PrismaClient()` inside feature files.
- ✖ Never deduct fee balances automatically without staff verification.
- ✖ Never generate batch PDFs on the main Next.js HTTP thread.

⚙️ Core Engineering & Code Hygiene Directives

- **Feature-Sliced Architecture:** Organize code strictly under `src/features/[feature]/` (components, service, schemas).
- **Dumb UI, Smart Services:** UI components only handle render/local state; all business logic lives in `*.service.ts`.
- **Uniform API Envelopes:** Enforce `{ success, data, error, metadata }` across all Route Handlers and Server Actions.
- **Universal Error Handling:** Use custom error classes (`AppError`, `UnauthorizedError`) with safe try/catch envelopes.
- **Strict Naming Standards:** PascalCase for components, camelCase for functions/hooks, snake_case for PostgreSQL entities.
- **Zod Environment Validation:** Validate all runtime variables at boot via `@/config/env.ts`; never read `process.env` directly.
- **Proactive Dead-Code Removal:** Eliminate unused imports, dead functions, and obsolete code during every feature cycle.

📋 Pre-Release Validation Checklist

- ✓ TypeScript compiles with zero errors (`tsc --noEmit`)
- ✓ Exit document barrier stops uncertified deletions
- ✓ Multi-branch context preserved across route changes
- ✓ UTR submission status stays PENDING until staff approval
- ✓ Attendance commits in single payload under 20ms
- ✓ Historical student report cards downloadable across years

06 Role-Based Access Control & Identity Matrix

Seven Non-Negotiable Institutional Clearances

🏛️ Super Admin (JSS Board)

Unrestricted trust-wide access; instant cross-campus switching; consolidated audit at `/all/all/analytics`; global fee policies.

🏫 Branch Admin (Principal)

Campus locked (`User.branchId`); campus admissions, staff substitution approvals, cycle promotion, and post-11 AM attendance override.

📚 Department Head (HOD)

Wing locked (`branchId + departmentId`); free teacher substitution assignment, wing leave approvals, and terminal grade book review.

👩‍🏫 Class Teacher (Faculty)

Classroom locked; submits 8:15–11:00 AM attendance register (single-payload `createMany`), reviews medical leaves, enters marks.

💼 Management (Bursar)

Bank statement UTR verification queue, fee ledger deductions, digital receipts, admission & exit certificate audit clearance.

👨‍👩‍👧 Parent / Guardian

Multi-ward family scoping; top navbar dropdown toggles active child context without re-auth; fee UTR payments, leave applications.

🎓 Student (Enrolled Pupil)

Personal profile view; timetable inspection, exam schedule view, fee dues, and **permanent lifetime access** to download official digital transcripts across all completed years.

FUNCTIONAL CAPABILITY	SUPER ADMIN	PRINCIPAL	DEPT HEAD	CLASS TEACHER	MANAGEMENT	PARENT / STUDENT
Cross-Branch Toggling	Unrestricted	Campus Locked	Campus Locked	Campus Locked	Campus Locked	Ward Locked
Class Attendance Submit	View / Audit	View / Override	View / Override	Submit Assigned	None	View Own
Attendance Lock Override	Full Override	Permitted (>11 AM)	With Audit Note	None (Frozen)	None	None
Fee Structure Config	Global Policy	Campus Level	None	None	Create / Modify	None

FUNCTIONAL CAPABILITY	SUPER ADMIN	PRINCIPAL	DEPT HEAD	CLASS TEACHER	MANAGEMENT	PARENT / STUDENT
UTR Verification & Ledger	Full Audit	View Only	None	None	Approve / Reject	Submit UTR
Student Leave Approvals	Full Override	Campus Level	Department Level	Assigned Class	None	Submit Leave
Batch Admit Card Printing	Permitted	Permitted	Permitted	None	None	None
Permanent Student Deletion	Sign-off Only	Execute (Exit Cert)	Blocked	Blocked	Blocked	Blocked
Lifetime Grade Downloads	Full Trust	Campus Level	Department Level	Class Level	None	View / Download